Capítulo 4. Modelado

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Todos los modelos son erróneos, pero algunos son útiles.*
>
> —George E.P. Box, estadístico británico (atribuido)

A estas alturas, debería quedar claro que la comunicación es un aspecto fundamental de tu trabajo como ingeniero de software, especialmente la comunicación entre tú y otros desarrolladores. Mientras que al ordenador solo le importa que el código sea sintácticamente correcto, la comunicación con otros seres humanos requiere mucho más. Tu código debe estar bien documentado y organizado para que otras personas puedan entenderlo (consulta el capítulo 3 para obtener más información sobre cómo escribir código), pero a veces querrás algo más. A lo largo de un proyecto, utilizarás modelos de software o diagramas de cajas y líneas para expresar tu intención técnica.

Al igual que un buen código, los buenos modelos de software son claros y fáciles de entender para las partes interesadas. Si tus modelos no

son claros, quienes los utilicen no comprenderán tu intención técnica. No faltan consumidores para tus modelos: usuarios, probadores, otros desarrolladores, personal de seguridad, personas que escriben los cheques, gestores de proyectos y arquitectos. Tú mismo. A veces, el único usuario de un diagrama serás tú. Dicho esto, no puedes esperar dibujar unos cuantos esquemas y que todo el mundo los entienda. Un diagrama que es perfecto para un desarrollador puede que no funcione tan bien para el vicepresidente de ingeniería. Y viceversa. Tu reto es saber qué diagrama crear y cuándo crearlo.

¿Qué es el modelado de software y por qué lo hacemos?

El software es una industria relativamente joven y, como tal, ha tomado prestados conceptos y enfoques de disciplinas más maduras.[1] También ha habido varias «olas» de enfoques de modelado, desde el Lenguaje Unificado de Modelado (UML) hasta el modelo C4. La industria de la construcción crea un conjunto completo de planos antes de comenzar un nuevo proyecto; ¿no deberían hacer lo mismo los proyectos de software? Quizás.

El modelado de software es el proceso de crear representaciones abstractas de un sistema de software para comprender, analizar y comunicar mejor su estructura, comportamiento y funcionalidad. Estos modelos guían a los desarrolladores, diseñadores y partes interesadas a través del proceso de diseño y desarrollo del sistema. Los buenos mo-

delos de software reflejan el ámbito de los problemas del mundo real, proporcionando información útil a lo largo del proceso de desarrollo.

Sin embargo, es importante comprender las diferencias fundamentales entre escribir software y construir una casa. Reestructurar el mundo físico es difícil y costoso: un constructor no puede permitirse probar la viabilidad de un muro de carga en producción, por así decirlo. Los planos permiten a los diseñadores asegurarse de que la casa cumple con los códigos de construcción locales y tiene todas las instalaciones adecuadas para el agua, la electricidad y la calefacción/refrigeración. Antes de que comience la construcción, los planos comunican lo que se va a construir, lo que permite al propietario aceptar que todo es como esperaba. Cuando la construcción ha terminado, los planos se pueden utilizar para confirmar que todo se ha construido según las especificaciones. Imagina el caos que se produciría en una obra si se dejara improvisar a todos los contratistas.

El software, sin embargo, se puede refactorizar.[2] El código se puede cambiar en unas pocas horas por unos cientos de dólares. El coste de «refactorizar» el mundo físico es considerablemente más alto; podría decirse que las casas se construirían de forma diferente si el coste de casi todos los cambios menores fuera de unas pocas horas extra y unos cientos de dólares. Los planos no serían tan esenciales para el proceso de construcción. Dado que el software es más maleable, es posible que los diagramas no sean tan críticos para el éxito del proyecto.

Los diagramas no se compilan; no dan como resultado un código ejecutable que contribuya a la finalización de tu proyecto. De hecho, se puede argumentar con bastante fuerza que, para los desarrolladores, tu

código es el artefacto de diseño definitivo. Jack Reeves, autor de los influyentes artículos Code as Design, sostiene que la programación es fundamentalmente una actividad de diseño y que la expresión más pura de ese diseño es el propio código.

Si los diagramas no se compilan, ¿por qué son útiles? Los diagramas pueden proporcionar contexto. Se pueden utilizar para comprender y gestionar la complejidad de un sistema y descomponer el problema. Puedes utilizarlos para predecir atributos de calidad, también conocidos como requisitos no funcionales o las capacidades que aprendiste en el capítulo 9. Los diagramas pueden ayudarte a diseñar para determinados atributos de calidad.

Algunas organizaciones exigen que crees varios diagramas como parte de tu SDLC. Estos requisitos pueden ser una bendición y una maldición.

Los diagramas pueden ayudarte a planificar el diseño de tu sistema. Si dibujar un esquema te ayuda a comprender un diseño, adelante, dibuja el diagrama. Una o dos horas esbozando una solución podrían ahorrarte días de tiempo de desarrollo.

Los diagramas también pueden ser útiles durante una expedición de arqueología de software, como cuando aprendes sobre un nuevo sistema o te expones a él por primera vez. Por supuesto, eso supone que esos diagramas sean precisos. Los diagramas pueden ser útiles a la hora de incorporar nuevos ingenieros. Una vez más, suponiendo que sean precisos. También pueden ser útiles a la hora de transferir conocimientos a un nuevo miembro del equipo, suponiendo, una vez más,

que sean precisos. Los diagramas pueden ser fundamentales a la hora de depurar un sistema, ya que conocer los límites del sistema y sus responsabilidades principales puede ayudarte a diseñar casos de prueba o a determinar cómo abordar la depuración.

Como ya habrás deducido, mucho depende de que tus diagramas sean precisos y estén actualizados. Si un diagrama no es claro o no representa el estado actual del código, su valor se ve muy mermado.[3] Esto lleva a una pregunta bastante interesante: ¿qué grado de permanencia tienen los diagramas? Se podría argumentar que los diagramas deberían tener una fecha de caducidad y que es perfectamente aceptable desechar un diagrama una vez que deja de ser útil. Pueden ser tan efímeros como un boceto en una pizarra. También pueden ser formales y estar realizados con una herramienta de modelado.

CONSEJO

Si te encuentras dibujando el mismo diagrama una y otra vez, es una buena indicación de que deberías formalizarlo de alguna manera. Tómate el tiempo necesario para crearlo con una herramienta y guárdalo en un lugar centralizado para tu proyecto; si te resulta útil, también lo será para tus compañeros de equipo. Para diagramas más grandes, utiliza una impresora plotter; una copia física colgada en la pared puede ser útil para todo el equipo. En algunos casos, imprimir un diagrama en cartulina puede hacer que resulte más real y valioso para las personas.

Por supuesto, también puedes simplemente tomar una foto del diagrama y añadirla a la documentación de tu proyecto, si lo prefieres. No dejes que una herramienta te frene.

No hay escasez de herramientas de modelado a tu disposición. Van desde simples herramientas de diagramación hasta herramientas de modelado empresarial de alta gama y con todas las funciones. Los proyectos masivos con grandes equipos repartidos por todo el mundo pueden encontrar un valor significativo en la estandarización de una herramienta empresarial determinada. Algunas herramientas sirven para dibujar modelos a mano, mientras que otras pueden generar modelos a partir del código.[4] Más adelante en este capítulo explorarás las herramientas, pero las herramientas sofisticadas no significan que vayas a obtener mejores diagramas que se adapten a tus necesidades.

Además de las herramientas, también hay muchos tipos de diagramas entre los que elegir.

¿Qué diagramas necesitas?

Cuando se trata de diagramas de modelado de software, hay una gran variedad entre los que elegir. El reto es saber qué diagrama crear y cuándo crearlo a lo largo del proceso de desarrollo del software. En otras palabras, ¿qué diagramas necesitas? ¿Cuáles son los más importantes en este momento del proyecto? Por supuesto, la única respuesta que podemos darte es: «Depende». ¿Qué estás tratando de hacer? ¿Qué tan complicado es el problema? ¿Qué tan riesgosa es la aplicación? ¿Alguna vez has creado una aplicación como esta o es algo nuevo? ¿Cuál es el presupuesto del proyecto? ¿Este proyecto es fundamental para el negocio?

Los diagramas pueden ser formales o informales. Los modelos formales utilizan notación técnica, como UML (Figura 4-1). Debes comprender al público al que va dirigido tu modelo. Cuanto más formal sea el modelo, más técnico deberá ser el público para poder utilizarlo. Esto suele significar que un subconjunto más reducido de personas comprenderá tu intención. Por ejemplo, ¿entiendes la notación Z?[5] Incluso si la entiendes perfectamente, ¿qué grado de difusión tiene dentro de tu organización?

Cuanto menos formal sea el método, menos conocimientos técnicos necesitará tener tu público para comprender lo que intentas comunicar. Esto permite que un público más amplio pueda comprender tu diagrama.

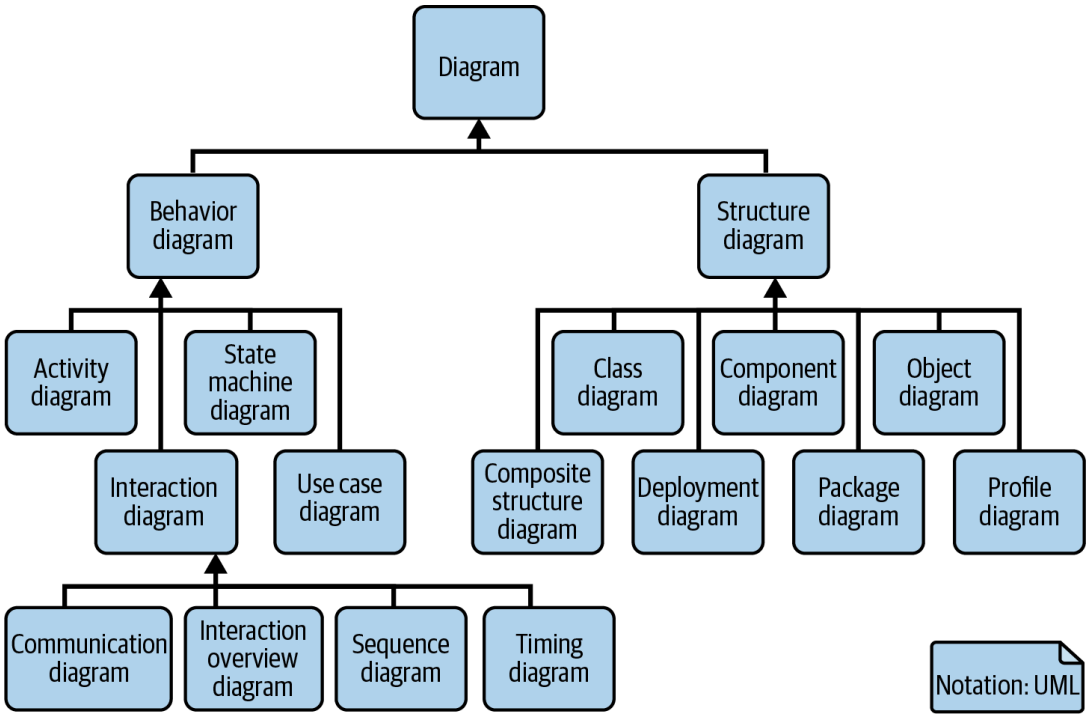


Figura 4-1. Diagramas UMLestándar , adaptados de Wikimedia Commons

LENGUAJE DE MODELADO UNIFICADO

El UML se desarrolló a mediados de la década de 1990 como una forma de armonizar otros sistemas de notación desarrollados en la década de 1980 y principios de la de 1990. El Object Management Group lo adoptó como estándar en 1997 y se publicó como norma ISO/IEC 19501 en 2005. Dicho esto, la mayoría de los ingenieros de software prefieren diagramas informales (que a menudo utilizan elementos UML). ¿Por qué?

¿Conoces la diferencia entre un rombo relleno y un rombo vacío en un diagrama UML?[6] Es posible que tú comprendas perfectamente el matiz de esta característica en particular. Sin embargo, es posible que tu público no lo haga. La mayoría de los ingenieros de software optan por utilizar la notación informal en lugar de UML con el fin de llegar a un público más amplio. No rechaces lo que funciona. Lo sencillo casi siempre es mejor.

Diagramas de contexto

Los diagramas de contexto definen los límites entre los sistemas o las partes de un sistema. Muestran el entorno y cómo interactúan las entidades. Se trata de entidades de datos lógicas que a menudo pueden incluir datos, al menos en términos de volumen y frecuencia. Los diagramas de contexto son de muy alto nivel.

Los diagramas de contexto son utilizados con frecuencia por arquitectos, directivos, gestores de proyectos y propietarios de productos. A

menudo se utilizan al principio de un proyecto para definir los límites y poner a todo el mundo de acuerdo sobre lo que se incluye y lo que no se incluye en una aplicación determinada. Proporcionan una visión general útil de lo que entendemos por un sistema determinado y muestran rápidamente el perímetro del mapa.

Piensa en los diagramas de contexto como la vista más alejada de un sistema. Por ejemplo, la figura 4-2 es un diagrama de contexto de un sistema que organiza y gestiona todos los datos de un coche autónomo. El coche envía datos que pueden ser analizados por científicos de datos y también envía notificaciones al propietario del vehículo.

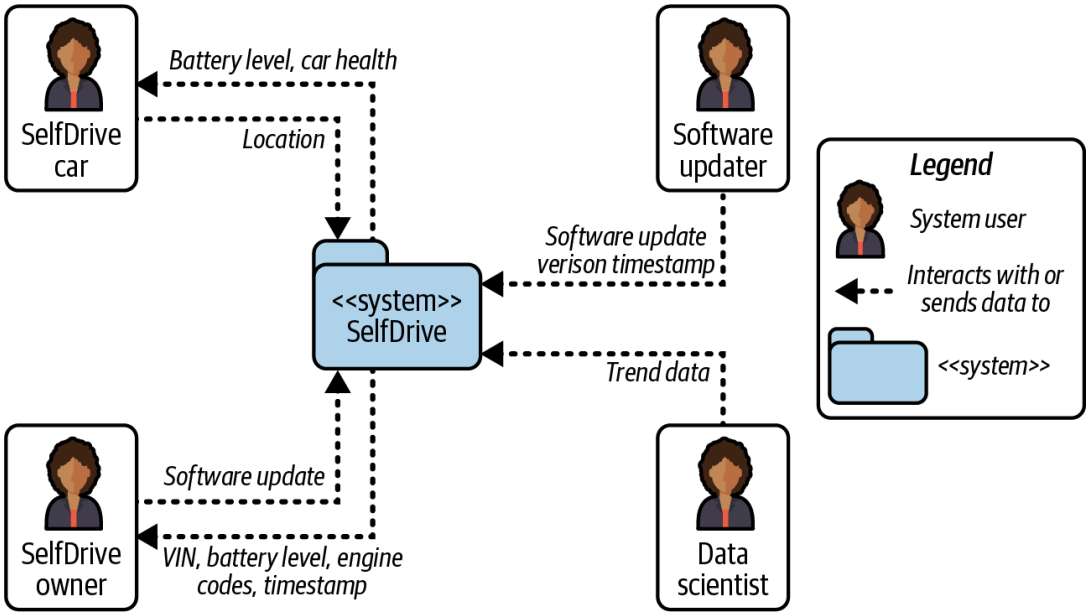


Figura 4-2. Ejemplo de diagrama de contexto

Diagramas de componentes

Los diagramas de componentes muestran los elementos principales de un sistema en tiempo de ejecución. Muestran cómo funciona el sistema en conjunto, ilustrando su estructura y comportamiento. Muestran los flujos de información y las interfaces. Los diagramas de componentes suelen ser utilizados por desarrolladores, arquitectos, personal de soporte de producción e ingenieros de DevOps.

Los diagramas de componentes se utilizan a lo largo de todo el proyecto. Originalmente, se diseñaron para definir las interacciones esperadas, pero también son útiles para explicar el proyecto a un público más técnico. Pueden ser muy útiles para la transferencia de conocimientos y la incorporación de nuevos empleados.

Por ejemplo, en la figura 4-3, tienes un sistema que actúa como un directorio de API disponibles. Hay dos vistas separadas, una específica para los administradores del sistema y otra para los consumidores y creadores de las API. También hay un directorio de usuarios para la autenticación y la autorización, y la información se almacena en una base de datos.

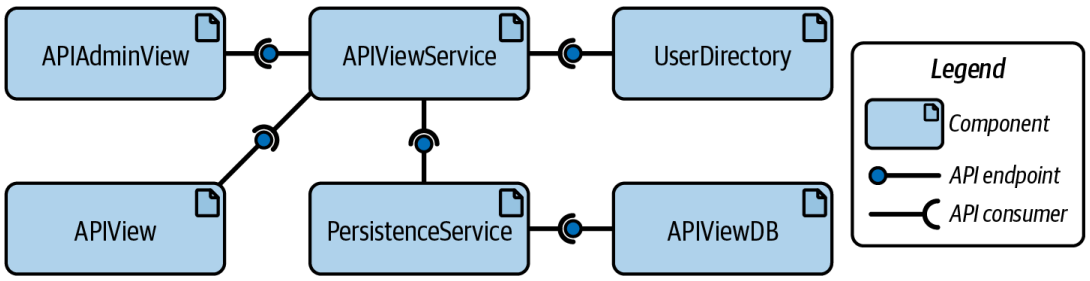


Figura 4-3. Ejemplo de diagrama de componentes

Diagramas de clases

Como su nombre indica,*los diagramas de clas es* muestran cómo se relacionan tus clases entre sí. Muestran las relaciones de herencia, así como las relaciones de composición, y pueden incluir una cardinalidad e .[7] Muestran entidades lógicas y pueden incluir métodos cuando sea útil.

Los diagramas de clases suelen extraerse del código existente según sea necesario, ya que, una vez creados, rápidamente quedan obsoletos con respecto al código. Pueden resultar abrumadores en sistemas grandes y pueden dividirse en límites lógicos o de dominio para que sean más fáciles de consumir.

El público destinatario de los diagramas de clases es de naturaleza muy técnica, normalmente otros ingenieros, arquitectos, desarrolladores, operadores, etc. Los diagramas de clases suelen crearse al principio de un proyecto para ayudar a comprender el panorama general y, por lo general, se perfeccionan a lo largo del proyecto. Pueden ser muy útiles tanto para los desarrolladores nuevos como para los ya existentes a la hora de comprender el alcance completo del sistema. Por ejemplo, el diagrama de clases de la figura 4-4 muestra que `Person` y `NamedEntity` son subclases de `BaseEntity`.

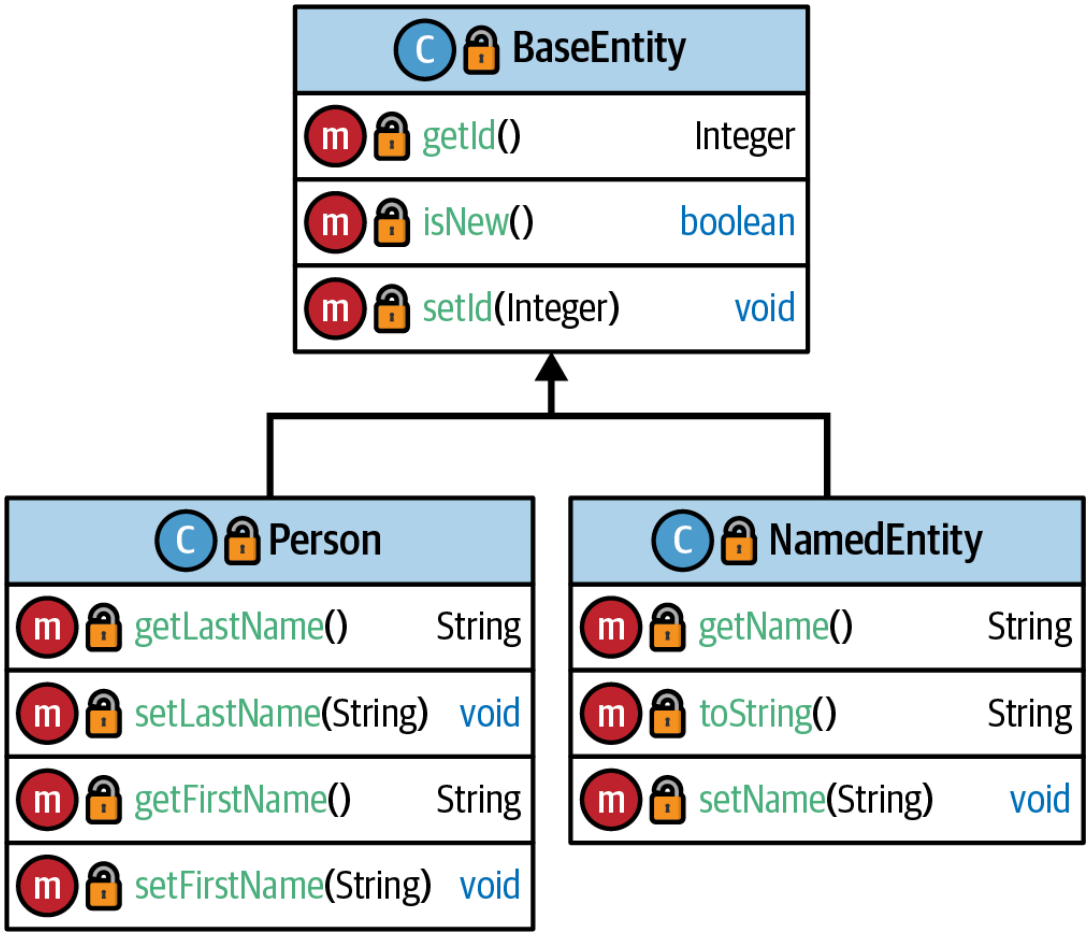


Figura 4-4. Ejemplo de diagrama de clases

Diagramas de secuencia

Los diagramas de secuencia muestran una secuencia de interacciones, aunque rara vez muestran todas las entidades o todas las interacciones. Por lo general, solo muestran las entidades que participan en un flujo concreto. La mayoría de los sistemas tienen interacciones casi ilimitadas y, por lo tanto, no hay una forma razonable de documentar todas las entidades y todos los flujos. Los diagramas de secuencia se utilizan con frecuencia para documentar las interacciones más interesantes o significativas desde el punto de vista arquitectónico, o como for-

ma de mostrar un patrón concreto o un uso estándar de una biblioteca.

Los diagramas de secuencia pueden mostrar operaciones, incluidos parámetros y tipos de retorno. Si bien esto puede ser útil para los desarrolladores, también es importante comprender que pueden quedar rápidamente obsoletos con respecto al código. Es posible extraer diagramas de secuencia del código existente, lo que puede ser útil al comenzar con una nueva base de código. Estos diagramas suelen ser creados por desarrolladores o arquitectos de soluciones. El público suele ser técnico, incluyendo desarrolladores, arquitectos y DevOps. El departamento de control de calidad también puede utilizar diagramas de secuencia para ayudarles a comprender qué deben probar.

Los diagramas de secuencia se utilizan a lo largo de todo el proyecto. Al principio del proyecto, definen un patrón, pero son increíblemente útiles para la transferencia de conocimientos y la incorporación. Por ejemplo, el diagrama de secuencia de la figura 4-5 vuelve al servicio API, mostrando que una búsqueda determina si el usuario está autorizado para realizar esa función y, en caso afirmativo, cómo fluye esa solicitud al almacén de datos y viceversa.

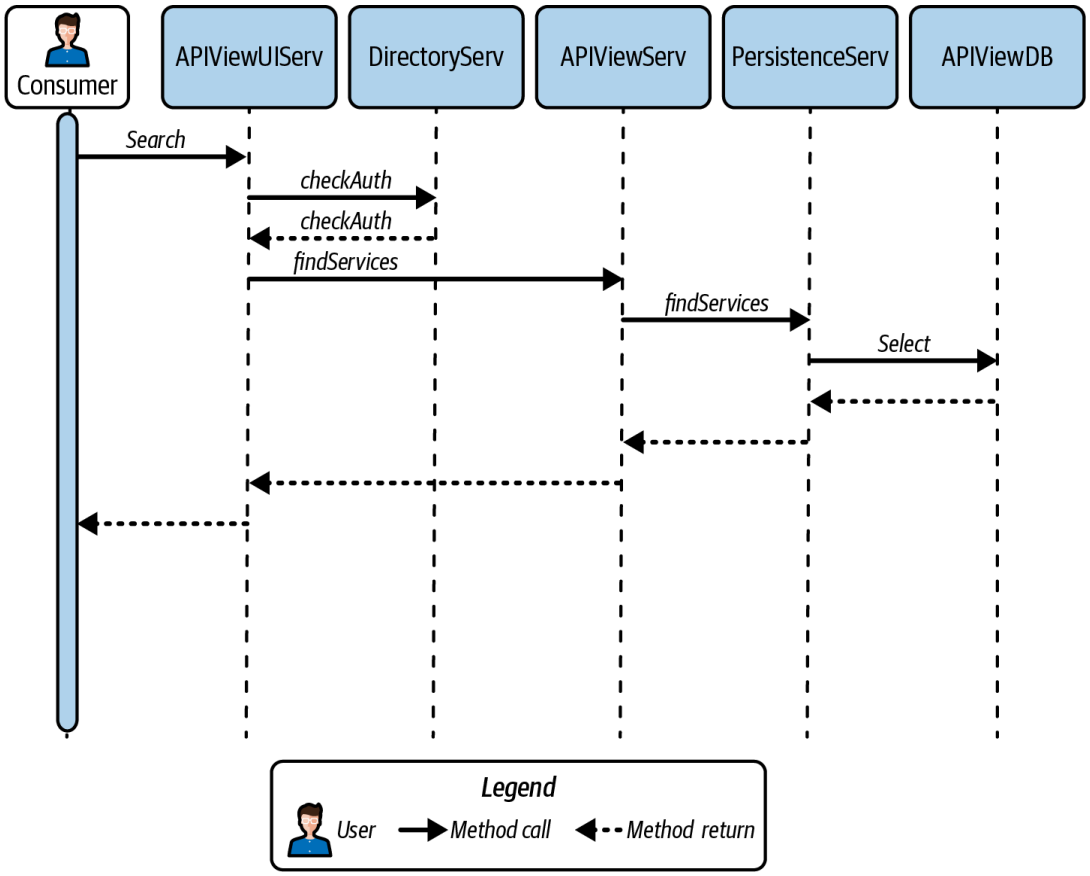


Figura 4-5. Ejemplo de diagrama de secuencia

Diagramas de implementación

Los diagramas de implementación proporcionan una vista en tiempo de ejecución de tu sistema. Muestran los nodos de hardware físicos, así como el software que se ejecuta en ellos. A menudo, muestran cómo está conectado el hardware y muestran los protocolos, así como la cardinalidad. Los diagramas de implementación están destinados a un público más técnico, como arquitectos, desarrolladores, DevOps y soporte de producción, así como infraestructura, arquitectos, seguridad y tu equipo de middleware.

Los diagramas de implementación pueden ser lógicos o físicos y deben etiquetarse adecuadamente. Por ejemplo, un diagrama de implementación lógico podría representar un clúster como una única entidad, mientras que un diagrama de implementación físico mostraría el número de un equilibrador de carga o servidor determinado que se ejecuta en un momento dado en dicho clúster.

Los diagramas de implementación se utilizan en las primeras fases de un proyecto para validar los atributos de calidad. Los sistemas que requieren asistencia las 24 horas del día, los 7 días de la semana, tendrán una implementación muy diferente a los que tienen requisitos de tiempo de actividad menos estrictos. Los diagramas de implementación pueden ayudarnos a comprender si una aplicación satisfará nuestras necesidades de escalabilidad, así como a validar la continuidad del negocio. También pueden ayudarnos a encontrar una implementación más rentable.

Muchas organizaciones tienen una plantilla estándar para sus implementaciones. La mayoría de las empresas tienen entornos de nube estándar o enfoques locales. En muchos sentidos, los diagramas de implementación se parecen un poco a los bloques de Lego. Es posible que tu empresa tenga un conjunto definido de herramientas y tecnologías que puedes utilizar, y que en su mayoría las encajes entre sí. Muchas organizaciones tienen arquitecturas de referencia estandarizadas que describen aplicaciones típicas y, al mismo tiempo, establecen algunos límites en tus opciones de implementación. Muy pocas empresas proporcionan una caja de herramientas ilimitada. Si tu empresa se ha estandarizado en Amazon Web Services (AWS), no crearías un diagrama lleno de entidades específicas de Azure, y si PostgreSQL fuera la base

de datos relacional aprobada por tu empresa, no querrías modelar una implementación utilizando MariaDB. Para ver un ejemplo de diagrama de implementación, consulta la Figura 4-6.

Este ejemplo explora el coche autónomo y muestra qué partes de la aplicación se implementan dentro de la zona segura y qué aspectos se encuentran fuera del firewall. Estos diagramas también pueden mostrar versiones específicas de los componentes.

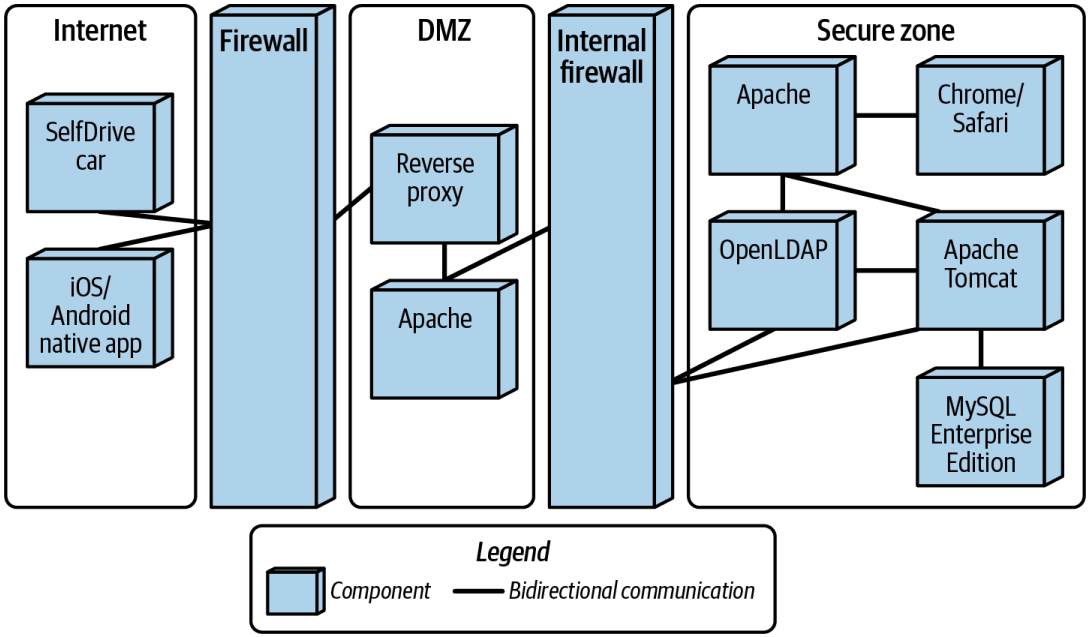


Figura 4-6. Ejemplo de diagrama de implementación

Es posible que te encuentres con una especialización de un diagrama de implementación conocido como *diagrama de seguridad*. Este modelo más detallado describe los mecanismos de seguridad de una aplicación. Los diagramas de seguridad suelen incluir protocolos y pueden aprovechar una vista de implementación o tecnología del sistema. Es-

tán destinados a un público más técnico, como desarrolladores, arquitectos y profesionales de la seguridad.

Los diagramas de seguridad se utilizan a lo largo de todo el proyecto. Pueden definir un patrón y validar que la solución cumple con las necesidades de seguridad del proyecto. Es importante tener en cuenta la información de identificación personal al interactuar con un sistema, así como cualquier normativa o ley que pueda aplicarse a tu sistema. Por ejemplo, véase la figura 4-7.

Una vez más, este ejemplo explora el coche autónomo, mostrando qué partes de la aplicación tienen su implementación dentro de la zona segura y qué aspectos se encuentran fuera del cortafuegos y deben tratarse como tales. Observe la incorporación de protocolos como HTTPS y LDAP.

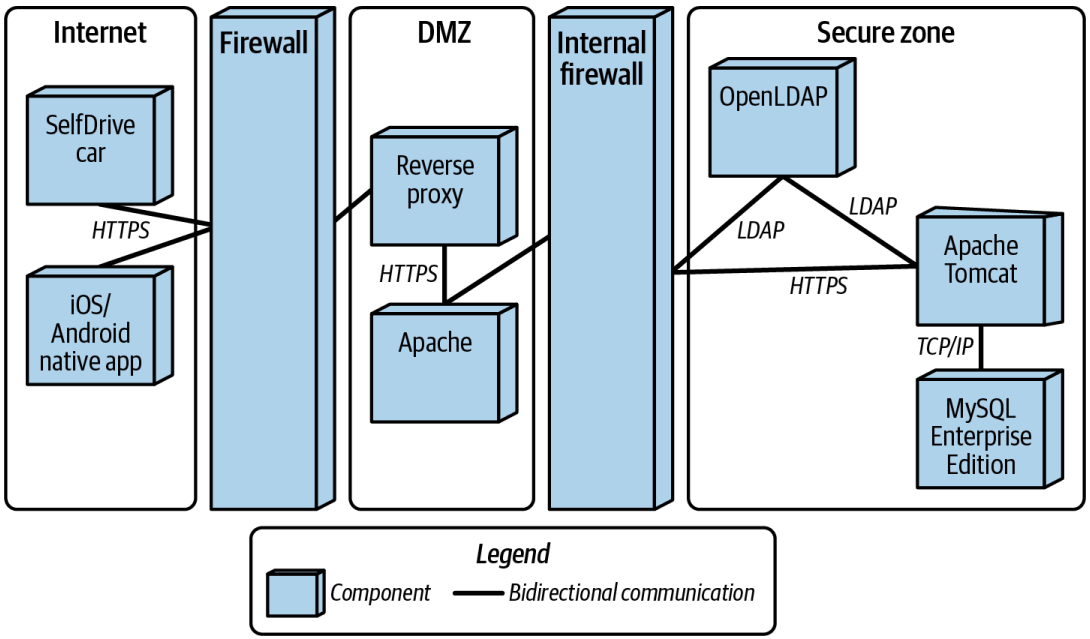


Figura 4-7. Diagrama de seguridadde ejemplo

Modelos de datos

Los modelos de datos muestran entidades de datos, así como relaciones. Pueden tener diferentes niveles de granularidad, desde más conceptuales hasta lógicos, hasta llegar al diseño físico del almacén de datos. Esencialmente, progresan desde un alto nivel hasta una implementación concreta. Los modelos conceptuales son de muy alto nivel y no están normalizados.[8] Los modelos lógicos muestran términos comerciales, a menudo en forma normal. Los modelos de datos físicos muestran detalles de implementación, incluidos los tipos de datos. Una vez más, progresan de menos detalles a más.

El público destinatario de los modelos de datos puede abarcar desde clientes hasta arquitectos de información y administradores de bases de datos, pasando por arquitectos de software, desarrolladores y personal de soporte. Los modelos de datos suelen crearse en una fase muy temprana del proyecto para ilustrar el dominio, aunque a menudo se perfeccionan a lo largo de la vida útil del proyecto. Por ejemplo, la figura 4-8 muestra algunas de las entidades de datos del sistema API. Una tecnología determinada tiene una o varias versiones, y un proveedor tiene una o varias tecnologías y plataformas.

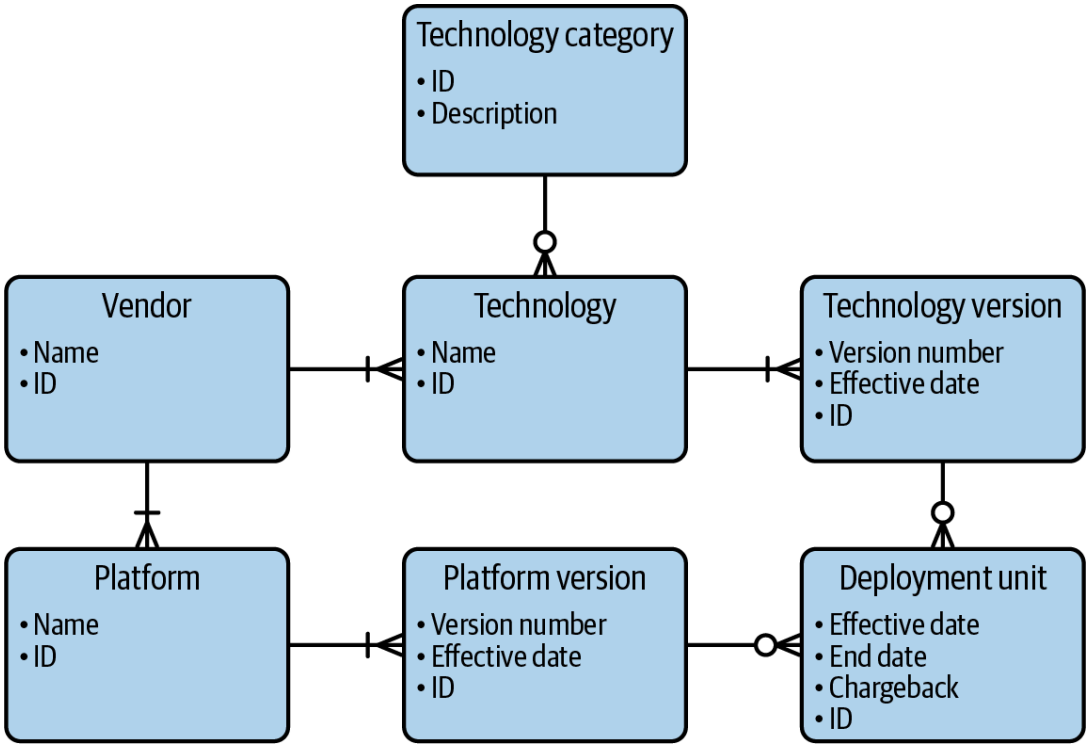


Figura 4-8. Diagrama de entidad-relaciónde ejemplo

Diagramas adicionales

Es posible que crees cualquier número de otros artefactos similares a diagramas durante la vida útil de un proyecto. Una vez más, en la mayoría de los casos, el proceso de creación puede ser muy valioso para comprender tu dominio, un problema específico o una interacción complicada. A menudo descubrirás que las personas no tienen una comprensión común del problema. Sin ningún orden en particular, aquí hay diagramas adicionales que puedes utilizar:

Tormenta de eventos

Técnica basada en talleres para explorar un ámbito empresarial. Se puede realizar de forma presencial o virtual, pero implica

que las partes interesadas empresariales y técnicas describan los eventos del ámbito y el comando que crea dichos eventos, así como el actor que ejecuta el comando. Las notas adhesivas de diferentes colores representan las distintas categorías. El Event Storming es un proceso muy flexible y ligero que se puede utilizar de diversas formas.

Mapeo de la cadena de valor

Una forma de analizar e ar el estado actual de un sistema, así como de modelar un estado final futuro. Se puede realizar de forma presencial o virtual. El mapeo de la cadena de valor te permite ver dónde hay fricciones o cuellos de botella en un proceso.

Mapeo de historias de usuario

Utiliza notas adhesivas junto con bocetos aproximados para trazar la experiencia del usuario de un sistema de software. Se puede realizar en persona o de forma virtual e implica que las partes interesadas, tanto empresariales como técnicas, trabajen juntas para explorar el sistema. Las tareas de alto nivel se representan como actividades, con los pasos y detalles necesarios detallados a continuación.

Modelos de recuperación ante desastres

Proporcionan una vista detallada de un sistema y ayudan a definir los requisitos de continuidad del negocio para tu proyecto. Es importante tener en cuenta el número de personas afectadas por una interrupción del servicio, así como el coste del tiempo de inactividad. Estos modelos muestran las entidades en tiempo de ejecución, así como el hardware en el que se ejecutan. Se utilizan a lo largo del ciclo de vida de un proyecto, aunque su

importancia aumenta a medida que se acerca el lanzamiento de la versión definitiva.

Buenas prácticas de modelado

Como se ha comentado, los modelos de « » pueden ser increíblemente útiles para comunicar intenciones técnicas. Un diagrama informal es relativamente rápido de dibujar, lo que puede ser tanto una bendición como una maldición. Los diagramas pueden salirse de control. A continuación se presentan algunas prácticas que pueden ayudarte a controlarlos.

Mantén la sencillez

Los diagramas pueden ser muy ruidosos. Es fácil crear un diagrama con el objetivo de impresionar y sorprender, en un intento de abrumar y desconcertar al público. Algunos diagramas están tan repletos de texto que te hacen preguntarte si necesitas revisarte la vista. Otros tienen elementos visuales que distraen, con líneas por todas partes y elementos de todas las formas y tamaños. Estos diagramas muestran tanto que, en última instancia, no comunican nada más que confusión.

Aunque son comunes, estos diagramas rara vez son útiles. Una vez más, *el objetivo de un diagrama es ayudarte a comunicar algo a los demás. Si alguien no puede descifrar tu diagrama, es inútil*. A medida que tu aplicación crezca, tendrás cada vez más entidades y más diagramas. ¿Deberías intentar mostrarlo todo en un solo diagrama? Aunque puede ser útil tener una visión general de tu aplicación, estos diagramas

suelen ser muy difíciles de entender. Si alguien lo imprimiera, ¿sería legible?[9]

No hay ninguna regla que diga que hay que incluir todo en un único diagrama. En muchos casos, es mejor dividir un diagrama para que los diagramas individuales sean más fáciles de entender. Un diagrama puede apuntar fácilmente a otro diagrama. Tomemos como ejemplo un diagrama de componentes. Si mostraras todos y cada uno de los componentes de tu aplicación, el diagrama se volvería rápidamente inutilizable. Sin embargo, normalmente puedes dividir el diagrama en puntos lógicos y hacer que un diagrama remita a otro. Es posible que sigas queriendo tener un diagrama general que muestre todos los componentes en una sola vista, pero rara vez debería ser tu único modelo del sistema.

Conoce a tu público

Al igual que al escribir código de software (véase el capítulo 3), los mejores ingenieros de software se centran en la persona que consume sus diagramas. Un diagrama muy técnico que es perfecto para un desarrollador puede no funcionar tan bien si se lo muestras a una persona sin conocimientos técnicos.

LOS DIAGRAMAS REQUIEREN CONTEXTO

Aquí Nate. Hace muchos años, fui el desarrollador principal de una nueva aplicación web. Como parte de mi trabajo en este proyecto, estaba muy orgulloso del diagrama de clases que había creado: a pesar de su complejidad, ¡no tenía líneas cruzadas! Concerté una cita con mi arquitecto para revisar mi trabajo. Con orgullo, le pasé mi diagrama por encima de la mesa para que lo revisara, pero él se encogió de hombros y me lo devolvió. Admito que me sentí un poco desconsolado; al fin y al cabo, no había líneas cruzadas. Aunque no esperaba una galleta por mi esfuerzo, sí que esperaba algún elogio.[10]

Lo que no entendía en ese momento era que mi arquitecto no tenía suficiente contexto para comprender mi diagrama de clases. Aunque esperaba que apreciara mi trabajo, él no tenía forma de identificar errores que no eran claros y obvios. No entendía nuestro ámbito empresarial al mismo nivel que yo en ese proyecto y, por lo tanto, no podía comentarlo de forma eficaz. Tenía que confiar en que mi trabajo era correcto. Eso no quiere decir que estos diagramas no sean útiles, solo que hay que entender sus límites.

Ten cuidado con la elección de los colores

A menudo utilizarás colores para designar ciertas cosas en tus diagramas. Por ejemplo, los componentes nuevos, modificados o sin tocar pueden tener colores diferentes. Si vas a utilizar colores, asegúrate de incluir una leyenda que identifique tus elecciones de color. Solo por-

que tu equipo utilice el verde para *lo nuevo*, no des por sentado que todos los demás también lo hacen.

Aunque no hayas impreso nada en años, debes asumir que alguien de tu organización al menos intentará imprimir tu diagrama. Un diagrama puede parecer fantástico en todo su esplendor cromático en tu monitor; sin embargo, una vez impreso en escala de grises, puede perder gran parte de su significado. También debes tener en cuenta que alguien de tu público puede ser daltónico. Moraleja de la historia: ten una alternativa a los colores.

Establece normas y plantillas

Puede ser muy útil definir un conjunto de normas dentro de tu organización. Un conjunto de diagramas que utilizas habitualmente junto con un conjunto de colores que esperas que te proporcionen coherencia en todos los proyectos. Simplemente no seas demasiado prescriptivo. Deja margen para añadir o eliminar diagramas según sea necesario en un proyecto determinado.

¿REALMENTE LO NECESITO?

Aquí Nate. Hace años, trabajé en una organización con un enfoque particularmente prescriptivo de la documentación de proyectos. Como arquitecto, se esperaba que creara cualquier número de diagramas; sin embargo, rápidamente descartaba lo que no necesitaba. Por ejemplo, nuestra plantilla en ese momento me obligaba a incluir un diagrama de casos de uso. Teniendo en cuenta que se trataba de un proyecto ágil que utilizaba historias y no casos de uso, me pareció extraño crear un diagrama solo por crear un diagrama. Así que no lo hice. No tengas miedo de rechazar una plantilla.

Herramientas

Es probable que tu organización ya disponga de una o dos herramientas de modelado. Sin embargo, puedes crear diagramas perfectamente adecuados sin necesidad de iniciar un largo proceso de adquisición.

Si tu empresa no dispone de ninguna, ten en cuenta lo siguiente (lista no exhaustiva):

- OmniGraffle es una herramienta de diagramación para MacOS e iOS de Omni Group. Incluye docenas y docenas de plantillas que te permiten crear rápidamente diagramas de software estándar. Con un entorno WYSIWYG de arrastrar y soltar, la curva de aprendizaje es relativamente baja.

- Microsoft Visio es una herramienta de diagramación para Windows. Visio te permite crear diversos diagramas, desde diagramas de flujo hasta mapas mentales, así como diagramas UML. Al igual que OmniGraffle, Visio utiliza un entorno WYSIWYG de arrastrar y soltar, lo que lo hace muy fácil de aprender. Si tu ordenador portátil de trabajo incluye la suite Microsoft Office, es probable que ya tengas Visio instalado. Hay varias plantillas descargables específicas para ingeniería de software y proveedores de nube pública como Azure, AWS y GCP.
- Mural, Miro y Lucidchart son herramientas basadas en navegador que actúan como una pizarra colaborativa distribuida. Te permiten crear cualquier número de diagramas. Varias personas pueden interactuar con el mismo lienzo simultáneamente, lo que las hace ideales para sesiones colaborativas.
- Mermaid es una herramienta de diagramación y creación de gráficos basada en JavaScript que utiliza definiciones de texto inspiradas en Markdown para crear y modificar diagramas de forma dinámica. Es especialmente popular para incrustar diagramas en documentación, wikis de GitHub y otras plataformas que admiten Markdown. Mermaid puede crear diagramas de flujo, diagramas de secuencia, diagramas de clases, diagramas de estado y mucho más utilizando una sintaxis de texto sencilla.
- Structurizr sigue el modelo de diagrama como código, lo que te permite crear cualquier número de diagramas utilizando el DSL de Structurizr. Aunque está diseñado para admitir el modelo C4, no se limita a esas visualizaciones e incluye temas para arquitecturas comunes basadas en la nube.

- PlantUML es compatible con muchos diagramas UML comunes, lo que te permite generar modelos a partir de archivos de texto a través de una interfaz de línea de comandos o GUI.
- Tu IDE. Es probable que tu IDE admita de forma nativa la creación directa de modelos, así como su extracción desde tu código base. Es posible que tengas que añadir uno o dos complementos, pero los editores actuales suelen incluir herramientas de modelado útiles.
- El software de presentación, como PowerPoint, Keynote y Google Slides, también se utiliza con mucha frecuencia, en gran parte debido a su omnipresencia en la mayoría de las empresas. Aunque no están diseñadas como herramientas de modelado, las plantillas específicas para la ingeniería de software pueden complementar las formas básicas incluidas por defecto. También pueden facilitar las sesiones colaborativas, ya que permiten que varias personas trabajen juntas en un diagrama.
- Las herramientas de IA son cada vez más comunes y muchas son capaces de extraer diagramas de una base de código. Sin embargo, siempre es posible que se produzcan alucinaciones, por lo que debes revisar cuidadosamente el resultado.

En caso de duda, elige una herramienta a la que todo el mundo tenga acceso y/o una licencia para utilizarla.

GENERACIÓN DE CÓDIGO A PARTIR DE MODELOS

Hace muchos años, algunas herramientas de modelado permitían generar código a partir de diagramas. Aunque esto podía resultar útil al iniciar un proyecto, estos enfoques han quedado relegados en gran medida al baúl de los recuerdos de la historia del software.[11] Esto se debe a que estas herramientas tenían un gran defecto: a menos que los desarrolladores observaran cuidadosamente las reglas de la herramienta, cualquier código que añadieran a los archivos generados solía borrarse la siguiente vez que se generaba código a partir del modelo.

Algunas herramientas funcionaban de forma bidireccional, al menos en teoría, modificando los modelos a medida que se actualizaba el código, y viceversa. Lamentablemente, no dieron lugar al enorme aumento de la productividad que prometían.

Independientemente de las herramientas que emplees, no olvides controlar las versiones de tus diagramas. Si bien las herramientas de diagramas como código son más adecuadas para el control de versiones, las herramientas modernas de control de versiones pueden manejar bastante bien los elementos visuales.

Aunque las herramientas más complejas suelen incluir funciones potentes, también suelen tener un precio elevado y una curva de aprendizaje prolongada. El uso regular puede remediar esto último, pero estas herramientas pueden complicar las cosas más de lo estrictamente necesario. No tengas miedo de utilizar una herramienta más sencilla si te permite transmitir tu mensaje de forma más fácil y rápida.

Sin embargo, ten en cuenta que el mensaje que intentas transmitir con tu diagrama es la parte más importante de la ecuación. La herramienta solo está ahí para ayudarte a expresar esa intención. Si te gusta una herramienta y te ayuda a ser más productivo, por supuesto que debes usarla. Sin embargo, no dejes que la herramienta te frene. El papel y el lápiz son excelentes herramientas de modelado, al igual que las pizarras blancas. En caso de duda, mantén la sencillez.

Recuerda que los diagramas no sustituyen al código funcional. Pueden ser muy útiles, pero también pueden quedar desactualizados con respecto al código. Algunos diagramas se pueden extraer del propio código, ya sea bajo demanda o como parte de un proceso de compilación, lo que puede aliviar el problema de la falta de sincronización. Sin embargo, siempre debes considerar el código como la fuente definitiva de la verdad.

Conclusión

Los modelos y diagramas son una parte importante del conjunto de herramientas de un ingeniero de software. Si bien el código es la fuente definitiva de verdad, los diagramas pueden ayudarte a comunicar conceptos clave a las partes interesadas técnicas y no técnicas; el reto es saber cuándo utilizar un enfoque concreto y cuándo omitirlo. Utiliza herramientas de modelado cuando y donde tengan sentido.

Ponerlo en práctica

Elige un flujo interesante en una aplicación que conozcas bien o, si te sientes valiente, una biblioteca de código abierto con la que estés familiarizado. Dibuja uno o dos diagramas de secuencia. Pide opinión a un compañero.

Extrae un diagrama de clases de tu aplicación. ¿Qué te sorprende de las relaciones? ¿Qué información te proporciona el modelo resultante sobre tu aplicación?

Pide a tu arquitecto que comparta los diagramas que haya creado para tus sistemas. Dedica algo de tiempo a familiarizarte con ellos. Si tienes alguna pregunta al respecto, ¡pregunta! ¿Qué harías de forma diferente si te pidieran que crearas uno de esos diagramas? Si hay algo que no te queda claro, coméntalo con tu arquitecto. Algunos proyectos generan diagramas como parte del proceso de CI/CD; si el tuyo no lo hace, quizá valga la pena añadirlo.

Recursos adicionales

- El modelo C4 para visualizar la arquitectura de software
- *User Story Mapping*, de Jeff Patton (O'Reilly, 2014)
- *Patrones de comunicación*, de Jacqui Read (O'Reilly, 2023)
- *Creating Software with Modern Diagramming Techniques*, de Ashley Peacock (Pragmatic Programmers, 2023)

- *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3.ª edición, de Martin Fowler (Addison-Wesley Professional, 2003)

1 Además de términos como *arquitecto* y *control de calidad*, en el ámbito de los datos te encontrarás con bibliotecarios, científicos y ontólogos.

2 Aunque puede ser más difícil y costoso de lo que algunos suponen.

3 Si están lo suficientemente desactualizados, se puede argumentar que los diagramas son activamente perjudiciales.

4 También es posible generar código a partir de modelos, algo que puedes encontrar principalmente en sistemas críticos para la seguridad.

5 Para ahorrarte una búsqueda en la web, *la notación Z* es un lenguaje de especificación formal desarrollado a finales de la década de 1970 y basado en las matemáticas.

6 Tiene que ver con las eliminaciones en cascada, pero tendrás que buscar el matiz.

7 *La cardinalidad* es el número de elementos de un conjunto.

8 *La normalización* es un proceso que reduce la redundancia y mejora la integridad de los datos.

9 Créenos, siempre hay alguien que quiere una copia impresa.

10 Vale, quizá esperaba una galleta.

11 Hoy en día, los desarrolladores solo crean soluciones de código, ¿verdad?